

Python Function

Functions are modular blocks of code designed to perform specific tasks. They enhance code efficiency and clarity by reducing code repetition and enabling code reuse.

Types of Functions in Python

Python primarily categorizes functions into two types:

Built-in Functions: These are predefined functions in Python that are readily available for use. Examples include `len()`, `range()`, and `abs()`.

User-defined Functions: As the name suggests, these are the functions defined by the users to perform specific tasks.

For a comparative understanding of functions in different programming languages, you can explore the [Types of Functions in C](#).

Python Function Declaration

In Python, a function is declared using the keyword `def`, succeeded by the name of the function and enclosed parentheses, which may enclose parameters. The syntax is as follows:

```
def function_name(parameters):  
    # function body
```

Function Name: This is the identifier for the function. It follows the same naming conventions as variables in Python. The function name should be descriptive enough to indicate what the function does.

Parameters (Optional): These are variables that accept values passed into the function. Parameters are optional; a function may have none. Inside the function, parameters behave like local variables.

Function Body: This block of code performs a specific task. It starts with a colon (`:`) and is indented. The function body typically contains at least one statement. The `return` statement can be used to send back a result from the function to the caller. `None` is returned if no `return` statement is used.

Creating a Function in Python

Creating a function in Python involves defining its structure and specifying the code it will execute. This process is straightforward and follows Python's emphasis on readability and simplicity.

Example:

```
def hello():  
    print("Hello World")
```

Calling a Function in Python

A function in Python is called by using its name followed by parentheses. In case parameters are present, these are included in the parentheses.

Example: To call the Function named hello, type the following:

```
def hello():  
    print("Hello World")  
  
hello()
```

Output:

Hello World

Learn more about calling a Python function [here](#).

Python Function with Parameters

Parameters are variables declared within the function definition. They act as placeholders for the values that are passed to the function when it is called. This allows a function to perform its task on different data inputs.

Syntax:

```
def function_name (parameters):
```

```
"""docstring """  
statement1  
statement 2  
...  
...  
return [expr]
```

Example:

```
def add_numbers(a, b):  
    """ Adds two numbers and returns the result. """  
    result = a + b  
    return result  
  
print(add_numbers(2,3))
```

Output:

5

Python Function Arguments

When you call a function in Python, the values you pass to it are arguments. These arguments are crucial for a function's operation, as they allow the passing of different data into the function, enabling it to perform operations on varied inputs.

Example:

```
def make_greeting(title, name, greeting="Hello"):  
    return f"{greeting}, {title} {name}!"
```

```
print(make_greeting("Dr", "Smith"))
```

Output:

Hello, Dr Smith!

Types of Python Function Arguments

Python supports several types of arguments:

Positional Arguments: These are the most common arguments, where the argument's value is based on its position. For example, in `func(a, b)`, `a` and `b` are positional arguments.

Keyword Arguments: The parameter name identifies these and can be supplied in any order, making your code more readable. For instance, `func(a=1, b=2)` explicitly states which parameter each argument corresponds to.

Default Arguments: A parameter with a default value can be omitted in the function call. The function will then use the default value.

Variable-length Arguments: You may need to determine the number of arguments supplied to your function. Python allows you to handle this with variable-length arguments, defined with an asterisk `*args` and `**kwargs`.

Example:

```
def make_greeting(title, name, greeting="Hello"):
```

```
    return f"{greeting}, {title} {name}!"
```

```
# Positional Arguments
```

```
print(make_greeting("Dr", "Smith"))
```

```
# Keyword Arguments
```

```
print(make_greeting(name="Alice", title="Prof"))
```

```
# Default Argument
```

```
print(make_greeting("Mr", "Brown"))
```

```
# All combined
```

```
print(make_greeting("Ms", name="Johnson", greeting="Hi"))
```

Output:

```
Hello, Dr Smith!
```

```
Hello, Prof Alice!
```

```
Hello, Mr Brown!
```

```
Hi, Ms Johnson!
```

Docstring

In Python, documentation strings, or docstrings, are literal strings used to document a Python module, function, class, or method. They are essential for understanding and maintaining code, as they provide a convenient way of associating documentation with Python code.

Syntax to print docstring is:

```
print(function_name.__doc__)
```

Example:

```
def add(a, b):
```

```
    """Add two numbers and return the result."""
```

```
    return a + b
```

```
print(add.__doc__)
```

Output:

Add two numbers and return the result.

Passing a List as an Argument

Here's a simple example of a function that takes a list as an argument:

```
def print_list(my_list):  
    for item in my_list:  
        print(item)  
  
print_list(["Happy ", "Coding"])
```

Output:

Happy
Coding

Python Function within Functions

In Python, functions can be defined inside other functions. These nested functions help organize code into more manageable pieces, encapsulating functionality or creating closures and decorators.

Example:

```
def outer_function(text):  
    def inner_function():  
        return text.upper()  
    return inner_function()
```

```
result = outer_function("hello")  
print(result)
```

Output:

HELLO

Anonymous Functions in Python

An anonymous in Python is a function that is defined without a name. Unlike functions defined using the `def` keyword, these are defined using the `lambda` keyword and are hence called lambda functions. They can have 0 or more arguments but only one return value.

Syntax:

lambda arguments: expression

Example:

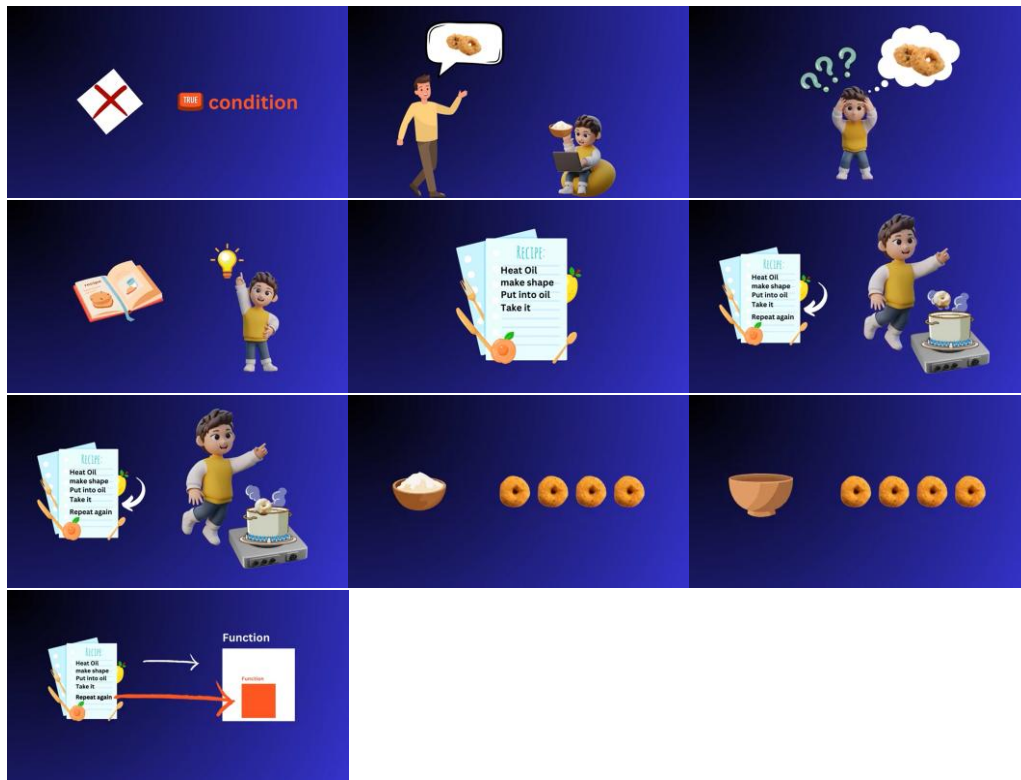
```
square = lambda x: x * x  
print(square(2))
```

The above example as a usually defined function could be written as:

```
def square(x):  
    return x * 2  
print(square(2))
```

Output:

Recursive Functions in Python



Recursion is the process in which a function calls itself. Recursive functions are used mostly for sequence generation and to make the code look clean and elegant.

Example of a function to calculate the sum of n consecutive natural numbers recursively:

```
def rec_sum(n):
    if n<=1:
        return n
    else:
        return n + rec_sum(n-1)
```



```
y = rec_sum(5)
print(y)
```

Output:

15

Python Library Functions

Python library functions are predefined functions available in Python libraries. These functions offer many functionalities without manual implementation, enhancing productivity and code efficiency.

Example:

```
import math
result = math.sqrt(9)
print(result)
```

Output:

3.0

Return Statement in Python

The return statement exits a function and returns to where it was called. This statement can optionally return a value from the function to the caller. If no expression is specified, None is returned.

Syntax:

```
return [expressions]
```

Example:

```
def add(a, b):  
    return a + b  
  
result = add(5, 3)  
print(result)
```

Output:

8

Pass by Reference and Pass by Value**Pass by Reference**

Concept: a reference (or pointer) to the actual data is passed in pass-by-reference. Modifications to the parameter within the function affect the passed argument.

Python's Approach: Python doesn't strictly follow this but behaves similarly with mutable objects. When a mutable object is passed, changes to it inside the function are reflected outside the function.

Pass by Value

Concept: Pass by value means the actual value is passed. The function works on a copy, and changes within the function do not affect the original data.

Python's Approach: With immutable objects (like integers and strings), Python's behaviour is akin to pass-by-value. Changes made to an immutable object in a function do not reflect in the original object.

Example:

```
def modify_elements(collection):  
    if isinstance(collection, list):  
        collection.append(100) # This will affect the passed list  
    else:
```

```
collection += 10 # This won't affect the passed integer
```

Mutable example

```
my_list = [1, 2, 3]
modify_elements(my_list)
print(my_list)
```

Immutable example

```
my_num = 7
modify_elements(my_num)
print(my_num)
```

Output:

```
[1, 2, 3, 100]
```

```
7
```

Benefits of Python Functions

Helps in increasing code modularity – Python functions help divide the code into smaller problems and solve them individually, thus making it easier to code.

Minimizes Redundancy – Python functions help you save the effort of rewriting the whole code. All you have to do is call the Function once it is defined.

Maximizes Code Reusability – Once defined, the Python Function can be called as many times as needed, thus enhancing code reusability.

Improves Clarity of Code – As the extensive program is divided into sections with the help of functions, it helps increase the readability of code while ensuring easy debugging.

Difference Between return and print in Python

Feature	return	print()
Purpose	Sends a value from a function	Displays output on the console
Data Usage	Can be stored in a variable	Cannot be stored (returns None)
Scope	Works only inside functions	Works anywhere in the script
Function Exit	Terminates function execution	Does not exit a function
Example Use Case	Used when you need to use the value later	Used for debugging or displaying results

Notes

Let's recall functions in Python in a nutshell:

Functions are used to reduce repetition and make the code more modular.

Python neither uses a pass-by-value nor a pass-by-reference mechanism, but it uses pass-by-object reference.

We can pass up to 255 parameters in a function in Python.

The scope of a variable in a function can be changed from local to global using the global keyword.

Pass statement can be used to write an empty function.

Scope of Variables in Python

The scope of a variable refers to the part of the program where a variable is accessible. In Python, variables can be local, global, nonlocal, or built-in.

Local Scope (Inside a Function)

A variable created inside a function can only be used inside that function.

It cannot be accessed outside the function.

Example:

```
def my_function():
```

```
    x = 10 # Local variable
```

```
    print("Inside function:", x)
```

```
my_function()
```

```
print(x) # This will cause an error (x is not accessible here)
```

What happens?

Inside the function: x = 10 works fine.

Outside the function: x is not defined.

Global Scope (Accessible Everywhere)

A variable created outside a function is global.

It can be used inside and outside functions.

Example:

```
x = 10 # Global variable
```

```
def my_function():
```

```
    print("Inside function:", x) # Can access global variable
```

```
my_function()
```

```
print("Outside function:", x) # Can access it here too
```

What happens?

x = 10 can be used anywhere in the program.

Global Keyword (Modify Global Variables)

Normally, a function cannot modify a global variable directly.

Use global to modify a global variable inside a function.

Example:

```
x = 5 # Global variable
```

```
def change_global():  
    global x # Declaring x as global  
    x = 10 # Changing global x  
    print("Inside function:", x)
```

```
change_global()  
print("Outside function:", x) # x is changed globally!
```

What happens?

x = 10 changes everywhere because we used global.

Nonlocal Scope (Inside Nested Functions)

If a function is inside another function, the inner function can access variables from the outer function.

Use nonlocal to modify a variable from the outer function.

Example:

```
def outer_function():  
    x = 5 # Variable in outer function  
  
    def inner_function():
```

```
nonlocal x # Use outer x
x = 10 # Modify outer x
print("Inside inner function:", x)
```

```
inner_function()
print("Inside outer function:", x) # x is changed
```

```
outer_function()
```

What happens?

x = 10 is changed inside both functions because we used nonlocal.

Built-in Scope (Python Reserved Keywords)

Python has built-in functions and keywords (like print(), len()).

You should not use these names as variable names.

Example:

```
print(len("Hello")) # Works fine
```

```
len = 5 # Bad idea! Overwrites built-in 'len'
print(len("Hello")) # Error!
```

What happens?

If you assign len = 5, it breaks the built-in len() function.

Table

Scope Type	Where It Works?	Can Be Modified Inside Function?	Keyword Needed?
Local	Inside the function only	Yes (Inside function)	No
Global	Everywhere (inside & outside functions)	No (unless using global)	global

Nonlocal	Inside nested functions	Yes (modifies outer function variable)	nonlocal
Built-in	Everywhere (default Python functions)	Not recommended	No